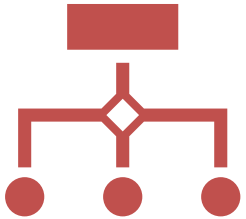
ReactJS.

The Component model (Contd)

Topics

- **More Hooks and Component Lifecycle.**
- **Data Flow patterns – Data Down, Action Up pattern.**
- **The Virtual DOM**

Sample App 1 – Version 2

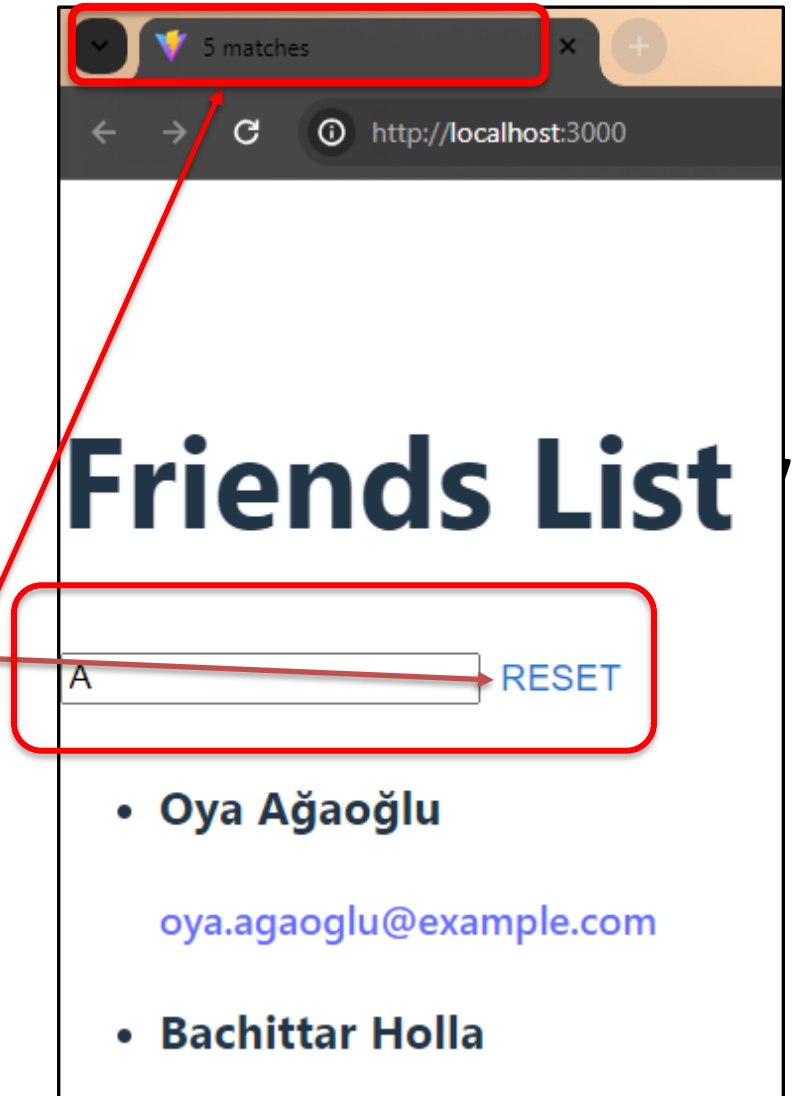


App UI changes:

A ‘Reset’ button – loads a new list of friends (overwriting the current list).

Combine Reset button and search text into a new component, SearchBox.

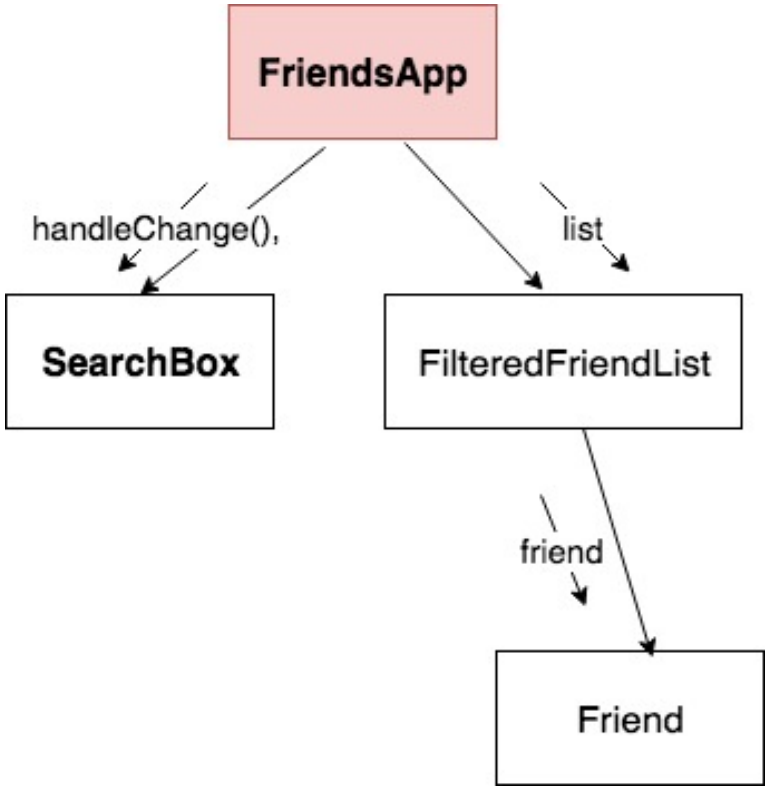
Browser tab title - shows # of matching friends (side effect).



Sample App 1 (v2) - Design

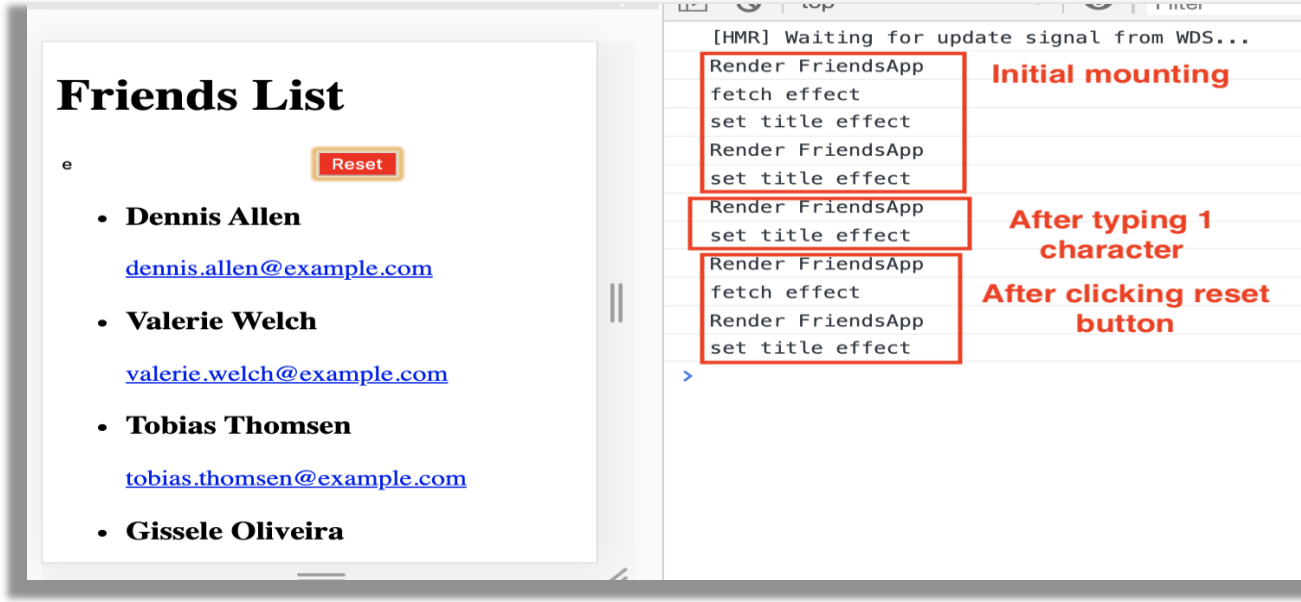
- **3 state variables:**
  1. **List of friends from API.**
  2. **Text box content.**
  3. *Reset button toggle.*

- **2 side effects, both with dependency arrays:**
  1. **'Fetch API data' -** reset button **toggle dependency.**
  2. **'Set browser tab title' - matching list <u>length</u> dependency.**



Sample App 1 (v2) - Events.

- **On mounting of** FriendsApp **component:**
  Both effects execute (Set browser tab to ’0 matches’).
  → **‘Fetch data’ effect initialises ‘friends list’ state.**
  → **Component re-renders → ‘Set browser tab’ effect executes.**
- **On typing a character in the text box:**
  ‘Text box’ state changes.
  → FriendsApp **rerenders + recomputes matching friends list**
  → **‘Set browser title’ effect executes.**
- **On clicking Reset button:**
  ‘Reset toggle’ state changes.
  → FriendsApp **re-renders.**
  → **‘Fetch data’ effect executes.**
  → **‘Friends list’ state changes.**
  → **FriendsApp re-renders + recomputes matching list**
  → **‘Set browser title’ effect executes.**



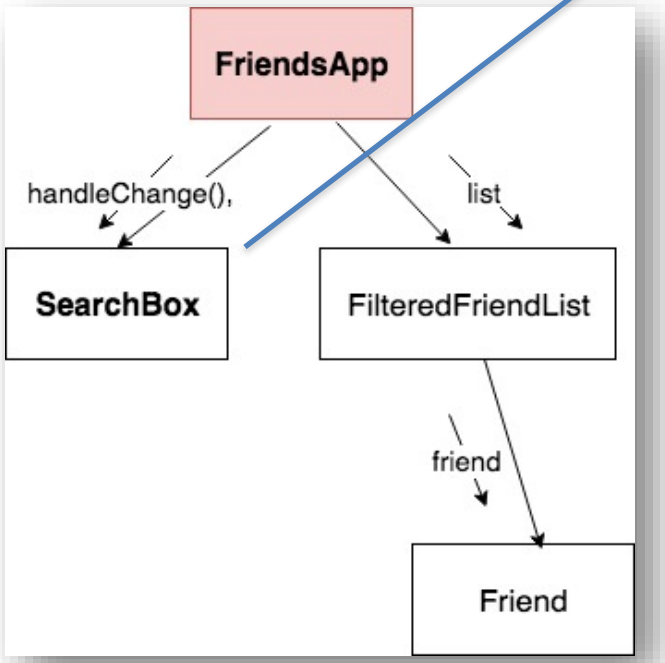
Topics

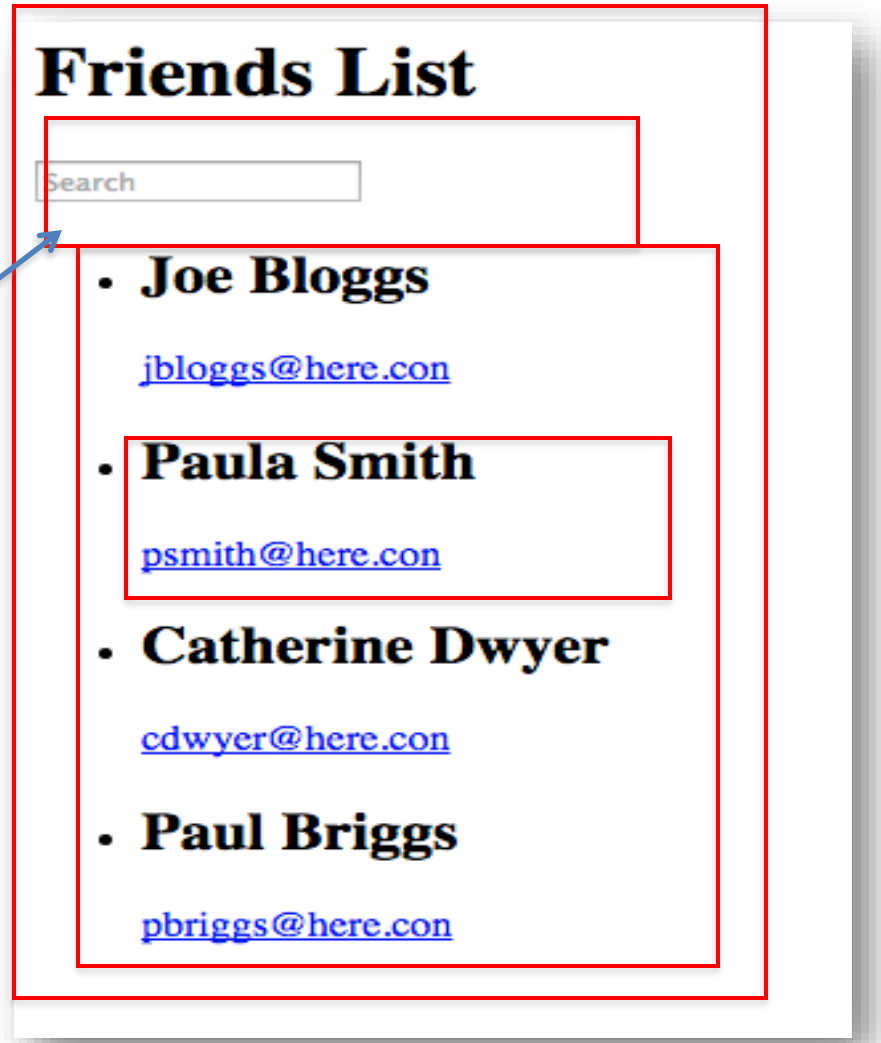
- **Hooks and Component Lifecycle.**
- **Data Flow patterns – Data Down, Action Up pattern.**
- **The Virtual DOM**

Sample App 2

(Data down, actions up pattern or Inverse data flow pattern)

- **Often a component's** state change **is caused by an** event **in a subordinate component.**
- **Solution: Use the** data down, action up pattern.





Data down, Action up.

Pattern:

1. Stateful **component provides a callback to the subordinate.**

2. Subordinate **invokes callback when the event occurs.**

```
const SearchBox = (props: SearchBoxProps) => {
  // Normalizes and forwards current input text to the parent.
  const onChange = (e: ChangeEvent<HTMLInputElement>) => {
    e.preventDefault();
    const newText = e.target.value.toLowerCase();
    props.handleChange(newText);
  };

  // Calls parent reset handler when the reset button is clicked.
  const onReset = (e: MouseEvent<HTMLButtonElement>) => {
    e.preventDefault();
    props.handleReset();
  };

  return <>
    <input type="text" placeholder="Search" onChange={onChange} />
    <Button onClick={onReset}>Reset</Button>
    </>
};
```

```
const FriendsApp = () => {
  const [searchText, setSearchText] = useState("");
  const [friends, setFriends] = useState([]);
  const [reset, setReset] = useState(false);

  useEffect(() => {
    console.log('fetch effect')
    fetch("https://fakerapi.it/api/v1/users?_quantity=10")
      .then(response => response.json())
      .then((json) => {
        setFriends(json.data);
      });
  },[reset]);

  // Updates the current search query used by the filter.
  const filterChange = (text: string) =>
    setSearchText(text.toLowerCase());

  // Toggles reset state to trigger a refetch of friend data.
  const toggleReset = () =>
    setReset(!reset);

  const filteredList = friends.filter((friend: friendProps) => {
    const friendName = friend.lastname.toLowerCase();
    return friendName.search(searchText) !== -1;
  });

  useEffect(() => {
    console.log('Set Title effect')
    document.title = `${filteredList.length} matches`;
  },[filteredList.length]);

  console.log("Render FriendsApp")
  return (
    <>
      <h1>Friends List</h1>
      <SearchBox handleChange={filterChange} handleReset={toggleReset} />
```

Handling Events in Code: React Event Handlers in Typescript

- Event handlers designed for value changes/mouse events etc.
- Event handler signature:
  - one argument, e, which has a corresponding generic type supplied by React Typescript libraries

event object that contains information about a change event on an HTML input element (e.g. textbox).

event object that contains information about a mouse event on an HTML button element.

```
const onChange = (e: ChangeEvent<HTMLInputElement>) => {
  e.preventDefault();
  const newText = e.target.value.toLowerCase();
  props.handleChange(newText);
};
```

```
const onReset = (e: MouseEvent<HTMLButtonElement>) => {
  e.preventDefault();
  props.handleReset();
};
```

```
return
<>
  <input type="text" placeholder="Search" onChange={onChange} />
  <Button onClick={onReset}>Reset</Button>
</>
```

Topics

- **Hooks and Component Lifecycle.**
- **Data Flow patterns – Data Down, Action Up pattern.**
- **The Virtual DOM**

Modifying the DOM

- **DOM – an internal data structure representing the browser's current 'display area' ; DOM always** in sync **with the display.**

- **Traditional performance best practice:**
  1. **Minimize direct accessing of the DOM.**
  2. **Avoid 'expensive' DOM operations.**
  3. **Update elements offline, then replace in the DOM.**
  4. **Avoid changing layouts in Javascript.**
  5. **. . . etc.**

- **Should the developer be responsible for low-level DOM optimization? Probably not.**
  - **React provides a** *Virtual DOM* **to shield developers from these concerns.**

The Virtual DOM

- **How React works:**
  1. **At app startup it create a lightweight, efficient form of the DOM, termed the *Virtual DOM*.**
  2. **The app changes the V. DOM whenever a component re-renders.**
  3. **When the re-rendering cycle is complete, the React engine:**
     1. **Performs a *diff* operation between the current and previous V. DOM instance.**
     2. **Computes the *set of changes* to apply to real DOM.**
     3. **Batch update the real DOM.**

- **Benefits:**
  a) **Cleaner, more descriptive programming model.**
  b) **Optimized DOM updates and reflows.**

Automatic Re-rendering (detail)

- **EX.: The Counter component.**

User clicks button

→ onClick event handler executed

→ component state is changed

→ component re-executed (re-renders)

→ The Virtual DOM has changed

→ React diffs the changes between the current and previous Virtual DOM

→ React batch updates the Real DOM

Re-rendering & the real DOM

- **What happens when the user types in the text box?**

  User types a character in text box
  → *onChange* ***event handler executes***
  → ***Handler changes a state variable***
  → ***React re-renders*** *FriendsApp component*
  → ***React re-renders children*** *(FilteredFriendList)* ***with new prop values.***
  → ***React re-renders children of*** *FilteredFriendList.*
  (Re-rendering completed)
  → ***(Pre-commit phase) React computes the updates required to the browser's DOM***
  → ***(Commit phase) React batch updates the DOM.***

  → ***Browser repaints screen***

Summary

- **A state variable change always causes a component to re-render.**
  - **State change logic is usually part of an event handler function.**
  - **Event hadler may be in a subordinate component.**

- **Side effects:**
  - **Always execute at mount time.**
  - **The dependency array will either reference a state variable, a value computed from a state variable, or a prop.**
    - **Can be multiple entries**
  - **Callback performs the side-effect, and may also cause a state change.**

- **Data flows downward, actions flow upward.**